YACHAY TECH UNIVERSITY
**School of Mathematical and Computational Sciences**
Course: Analysis of Algorithms | Lab 07

# Assignment 07: Gaussian Elimination, Heaps, and LU Decomposition

**Author:** Darío Pomasqui | **Date:** October 19, 2026

**GitHub Repository:**
https://github.com/darioxx32/Algorithm-analysis/tree/main/Deber7

**Abstract**

This report develops and analyzes the four algorithms requested in Assignment 07. The first program converts an augmented matrix to upper-triangular form by Gaussian forward elimination with partial pivoting. The second solves an upper-triangular system by backward substitution and derives its quadratic running time. The third transforms an arbitrary array into a max-heap using bottom-up heap construction. The fourth derives an LU factorization from Gaussian elimination, including row permutations, and solves a linear system using forward and backward substitution. The C programs were compiled with GCC under C11 with warnings enabled; console captures record the resulting program executions.

**Conventions.** Matrices are indexed from zero in C and from one in the mathematical notation when convenient. The linear-system examples use the same system

$$\begin{aligned} 2x + y - z &= 8, \\ -3x - y + 2z &= -11, \\ -2x + y + 2z &= -3, \end{aligned} \qquad \text{whose solution is } (x, y, z) = (2, 3, -1).$$

Using the same input in Exercises 1, 2, and 4 makes it possible to compare the triangular system, its back-substitution result, and the LU solve.

## Exercise 1. Forward elimination for Gaussian elimination

### 1.1 Goal and mathematical operation

For a system $Ax = b$, Gaussian elimination applies elementary row operations to the augmented matrix $[A \mid b]$ until it is upper triangular. At pivot column $k$, let $p$ be the selected pivot row. For each row $i > k$, define the multiplier

$$m_{ik} = \frac{a_{ik}}{a_{kk}}.$$

Subtracting $m_{ik}$ times row $k$ from row $i$ changes the entries by

$$a_{ij} \leftarrow a_{ij} - m_{ik}a_{kj}, \qquad j = k+1, \ldots, n,$$

where the last column is the right-hand side. This makes the entry below the pivot zero while preserving the solution set, provided the same operation is performed on the augmented column.

### 1.2 Pivoting and implementation

The program uses partial pivoting: among rows $k$ through $n-1$, it picks the row whose entry in column $k$ has the largest absolute value, then swaps that row with row $k$. This avoids dividing by a zero pivot whenever a valid pivot exists and generally reduces numerical error compared with always using the diagonal row. A tolerance of $10^{-12}$ is used to detect a pivot too close to zero for this small demonstration. If no usable pivot exists, the program reports that a unique solution cannot be obtained.

For the sample matrix, the first pivot is $-3$, since it has the greatest magnitude in the first column. The first elimination step removes the $-2$ and 2 entries below it; after the next pivot step the output is an upper-triangular system. Forward elimination costs $\Theta(n^3)$ arithmetic operations for a dense $n \times n$ system: there are $O(n)$ pivot columns, $O(n)$ rows below each pivot, and up to $O(n)$ entries updated in each row. The matrix is stored in-place, so the auxiliary storage beyond the input is $O(1)$ (the input itself occupies $O(n^2)$).

**Correctness invariant.** At the start of iteration $k$, all entries below the diagonal in columns $0, \ldots, k-1$ are zero. Eliminating column $k$ preserves those earlier zeros and makes all entries below the new pivot zero. Thus, after the final pivot column, the coefficient matrix is upper triangular and the row operations have preserved the solutions of the original system.

```c
#include <stdio.h>
#include <math.h>
#define N 3
int main(void) {
    double a[N][N+1] = {{2,1,-1,8},{-3,-1,2,-11},{-2,1,2,-3}};
    for (int k=0;k<N-1;k++) {
        int p=k;
        for (int i=k+1;i<N;i++) if (fabs(a[i][k])>fabs(a[p][k])) p=i;
        if (fabs(a[p][k])<1e-12) { puts("No unique solution."); return 1; }
        if (p!=k) for (int j=k;j<=N;j++){double t=a[k][j];a[k][j]=a[p][j];a[p][j]=t;}
```

```c
            for (int i=k+1;i<N;i++) { double m=a[i][k]/a[k][k]; a[i][k]=0;
                for (int j=k+1;j<=N;j++) a[i][j]-=m*a[k][j]; }
        }
        if (fabs(a[N-1][N-1])<1e-12) { puts("No unique solution."); return 1; }
        puts("Upper triangular system:");
        for(int i=0;i<N;i++) printf("%.2f %.2f %.2f | %.2f\n",a[i][0],a[i][1],a[i][2],a[i][N]);
        return 0;
}
```

Listing 1: gaussian.c: forward elimination with partial pivoting

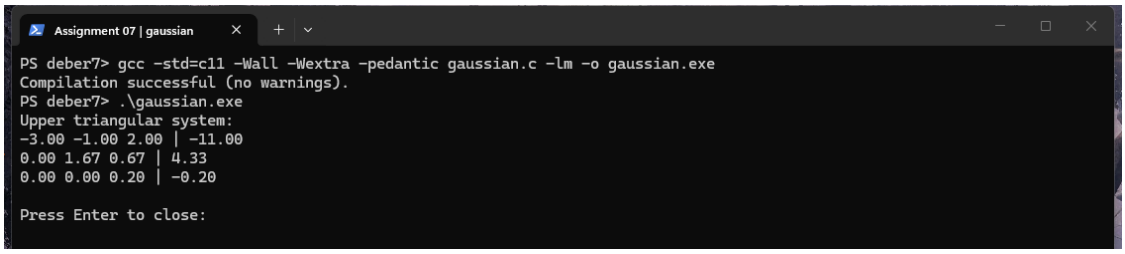



Figure 1: GCC compilation command and subsequent execution of `gaussian.exe`; the output is the upper-triangular augmented system.

# Exercise 2. Backward substitution and its running time

## 2.1 Algorithm

After forward elimination, the system has the form $Ux = c$, where $U$ is upper triangular. The last equation contains only $x_{n-1}$, so solve it first and then proceed upward:

$$x_i = \frac{c_i - \sum_{j=i+1}^{n-1} u_{ij} x_j}{u_{ii}}, \qquad i = n-1, n-2, \ldots, 0.$$

At each row, all variables with index greater than $i$ have already been computed. The code stores the resulting vector in $x$ and uses the known triangular matrix and right-hand side.

## 2.2 Proof of the quadratic running time

For row $i$, the summation has $n - i - 1$ terms. Counting each multiply/subtract pair as constant work, the total work over all rows is

$$\sum_{i=0}^{n-1} (n - i - 1) = \sum_{r=0}^{n-1} r = \frac{n(n-1)}{2}.$$

There are also $n$ divisions and $O(n)$ other operations. Hence the total running time is $\Theta(n^2)$, which establishes the requested $O(n^2)$ upper bound. The output vector

requires $O(n)$ extra space. The cost assumes every diagonal entry $u_{ii}$ is nonzero; a zero diagonal pivot indicates that the system cannot be solved by this routine without pivoting or further rank analysis.

For the triangular system in Exercise 1, backward substitution computes $z = -1$ from the last row, then $y = 3$, and finally $x = 2$. This recovers the solution of the original system.

```c
#include <stdio.h>
#define N 3
int main(void) {
    const double u[N][N]={{2,1,-1},{0,.5,.5},{0,0,-1}};
    const double b[N]={8,1,1};
    double x[N];
    for(int i=N-1;i>=0;i--){double s=b[i];for(int j=i+1;j<N;j++)s
    -=u[i][j]*x[j];x[i]=s/u[i][i];}
    printf("Solution: x = %.2f, y = %.2f, z = %.2f\n",x[0],x[1],x
    [2]);
    return 0;
}
```

Listing 2: back_substitution.c: solve an upper-triangular system

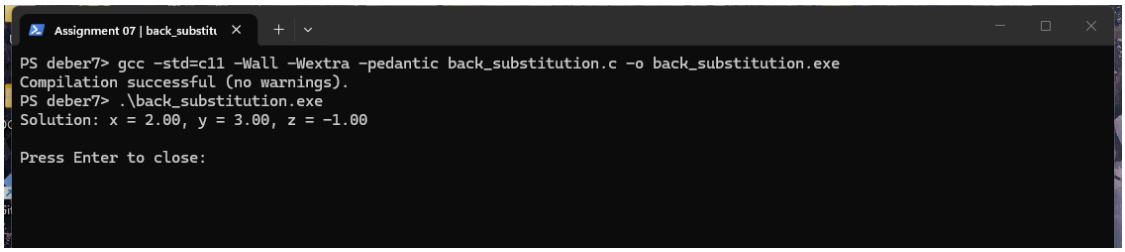



Figure 2: GCC compilation command followed by execution of `back_substitution.exe` and its solution.

# Exercise 3. Transforming an array into a max-heap

## 3.1 Heap property and representation

A binary max-heap is a complete binary tree in which every parent key is at least as large as either child key. The program stores that tree in an array in level order. With zero-based indexing, node $i$ has left child $2i + 1$ and right child $2i + 2$. The heap need not be sorted: it only guarantees that the maximum value is at the root and that each parent dominates its children.

## 3.2 Bottom-up construction

Leaves already satisfy the heap property, so the algorithm starts at the last internal node $\lfloor n/2 \rfloor - 1$ and calls `heapify` on each node moving toward the root. The iterative `heapify` compares a node with its children, swaps it with the larger child when needed, and continues down that subtree. When it stops, the subtree rooted at that node is a

max-heap. Because children are processed before their parents, each call can rely on both child subtrees already being heaps.

A call to `heapify` can move down at most the height of the tree, so a single call costs $O(\log n)$. However, applying that bound to all $n/2$ internal nodes would be loose. Nodes close to the leaves have height zero or one, and only a few nodes have large height. The aggregate work is bounded by

$$\sum_{h=0}^{\lfloor \log_2 n \rfloor} O\left(\frac{n}{2^{h+1}}\, h\right) = O(n),$$

so bottom-up heap construction is linear-time. The iterative procedure uses $O(1)$ auxiliary space in addition to the input array.

**Result check.** The output array is a valid max-heap when each internal array position is at least as large as its existing children. Its root is the largest array value. The relative order of siblings or of different subtrees is not specified.

```c
#include <stdio.h>
#define N 10
void heapify(int a[],int n,int i){
    while(1){int largest=i,l=2*i+1,r=2*i+2;
        if(l<n&&a[l]>a[largest])largest=l;
        if(r<n&&a[r]>a[largest])largest=r;
        if(largest==i)return;
        int t=a[i];a[i]=a[largest];a[largest]=t;i=largest;
    }
}
int main(void){int a[N]={4,10,3,5,1,8,2,7,6,9};
    for(int i=N/2-1;i>=0;i--)heapify(a,N,i);
    printf("Max-heap array: ");for(int i=0;i<N;i++)printf("%d%s",a
    [i],i==N-1?"\n":" ");return 0;
}
```

Listing 3: heap.c: bottom-up max-heap construction

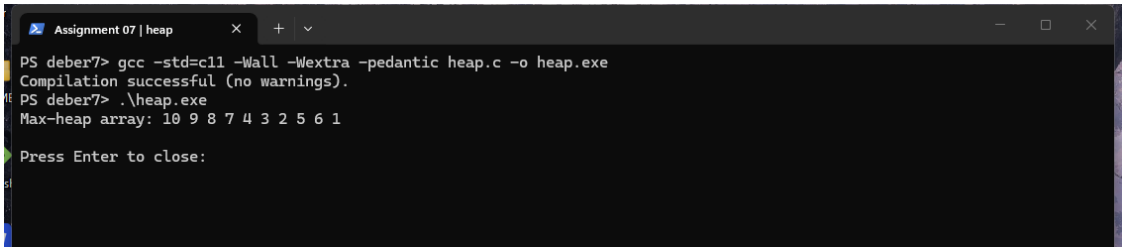



Figure 3: GCC compilation command followed by execution of `heap.exe` and its max-heap array.

## Exercise 4. Solving a system using LU decomposition

### 4.1 From elimination to factorization

Gaussian elimination can be recorded as a factorization instead of discarding the elimination operations. The multipliers used to eliminate entries below each pivot become the subdiagonal entries of a lower-triangular matrix $L$; the final eliminated matrix becomes the upper-triangular matrix $U$. With partial pivoting, row swaps must also be recorded in a permutation matrix $P$, giving

$$PA = LU.$$

The diagonal of $L$ is set to one. The equation $Ax = b$ is then transformed by applying $P$:

$$LUx = Pb.$$

Define $y = Ux$. First solve $Ly = Pb$ using forward substitution, then solve $Ux = y$ using backward substitution.

### 4.2 Row swaps and algorithm details

In `lu.c`, the permutation vector records which original row occupies each current row. When a pivot swap occurs, the program swaps rows of the working matrix, swaps already computed entries in the corresponding rows of $L$, and updates the permutation vector. Swapping the previous columns of $L$ is essential: those entries contain multipliers from earlier elimination steps and must follow their rows. The program then performs elimination, storing each multiplier in $L$ and updating the remaining working matrix to form $U$. If a pivot is smaller than the tolerance, it reports a singular matrix.

The factorization takes $\Theta(n^3)$ arithmetic operations for a dense matrix. Once factors are available, each triangular solve takes $\Theta(n^2)$, so solving one system after factorization costs $\Theta(n^3)$ overall. If the same matrix is reused with several right-hand sides, the factorization is done once and each additional solve costs $\Theta(n^2)$. The program requires $O(n^2)$ storage for the matrices and $O(n)$ storage for vectors and the permutation.

The example's first column causes a pivot-row swap, illustrating why the permutation $P$ is needed. The resulting solution printed by the program is $x_1 = 2$, $x_2 = 3$, and $x_3 = -1$.

```c
#include <stdio.h>
#include <math.h>
#define N 3
int main(void){
    double a[N][N]={{2,1,-1},{-3,-1,2},{-2,1,2}};
    double b[N]={8,-11,-3}, l[N][N]={{0}}, u[N][N]={{0}}, y[N], x[N];
    int p[N]={0,1,2};
    for(int i=0;i<N;i++)l[i][i]=1;
    for(int k=0;k<N;k++){
        int q=k;for(int i=k+1;i<N;i++)if(fabs(a[i][k])>fabs(a[q][k]))q=i;
        if(fabs(a[q][k])<1e-12){puts("Singular matrix.");return 1;}
        if(q!=k){for(int j=0;j<N;j++){double t=a[k][j];a[k][j]=a[q][j];a[q][j]=t;}
```

```
13              for(int j=0;j<k;j++){double t=l[k][j];l[k][j]=l[q][j];
    l[q][j]=t;}
14              int t=p[k];p[k]=p[q];p[q]=t;}
15          for(int j=k;j<N;j++)u[k][j]=a[k][j];
16          for(int i=k+1;i<N;i++){l[i][k]=a[i][k]/u[k][k];for(int j=k
    ;j<N;j++)a[i][j]-=l[i][k]*u[k][j];}
17       }
18       for(int i=0;i<N;i++){double s=b[p[i]];for(int j=0;j<i;j++)s-=l
    [i][j]*y[j];y[i]=s;}
19       for(int i=N-1;i>=0;i--){double s=y[i];for(int j=i+1;j<N;j++)s
    -=u[i][j]*x[j];x[i]=s/u[i][i];}
20       puts("Solution from PA=LU:");for(int i=0;i<N;i++)printf("x%d =
     %.2f\n",i+1,x[i]);return 0;
21   }
```

Listing 4: lu.c: Gaussian-elimination-based $PA = LU$ factorization and solve

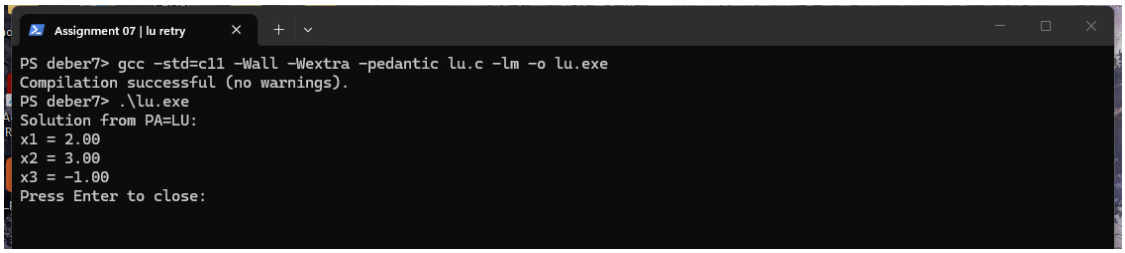



Figure 4: GCC compilation command followed by execution of `lu.exe` and its solution.

## Compilation, execution, and conclusion

Each source file is a standalone C11 program. The programs were compiled with GCC using warnings enabled. For example:

```
gcc -std=c11 -Wall -Wextra -pedantic gaussian.c -lm -o gaussian.exe
.\gaussian.exe
```

The `-lm` library option is required by the programs that use `fabs`. All four programs compiled without warnings and their outputs are shown in the included console captures. The implementations demonstrate the connections between elimination and triangular solving, the linear construction time of a heap, and the way Gaussian elimination yields an LU factorization. Backward substitution requires quadratic time because its row sums contain $0, 1, \ldots, n-1$ terms in total.